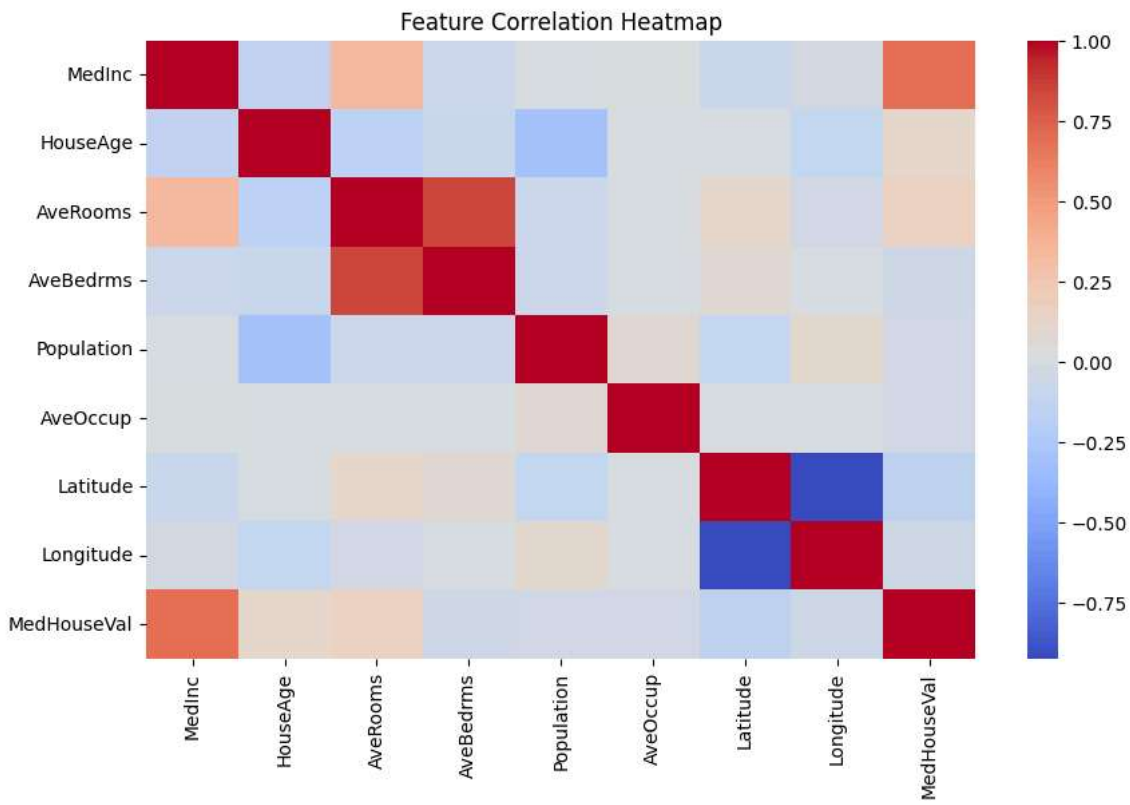
Practical 14: ML Model on Boston Housing Dataset## Practical Information| Attribute | Details ||----------|---------|| **Practical Number** | 14 || **Title** | ML Model on Boston Housing Dataset || **Duration** | 2 Hours || **Course Outcomes (COs)** | CO4, CO5 || **Description |** Build an end-to-end ML pipeline with the Boston Housing dataset. |## Key Topics Covered- Complete ML pipeline- EDA- Feature engineering- Model comparison- Deployment---## Instructions for Students1. **Complete all tasks** outlined in this practical2. **Write your code** in the cells provided below3. **Add comments** to explain your code4. **Include proper output** (prints, visualizations, etc.)5. **Submit your completed notebook** before the deadline---## Your Work Starts HereUse the cell(s) below to write your implementation:

```
# Practical 14 - Code 1: EDA & Preprocessing

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load dataset
data = fetch_california_housing(as_frame=True)
df = data.frame

print("Dataset Shape:", df.shape)
print("\nFirst 5 Rows:\n", df.head())
print("\nMissing Values:\n", df.isnull().sum())

# Correlation heatmap
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title("Feature Correlation Heatmap")
plt.show()

# Split features & target
X = df.drop("MedHouseVal", axis=1)
y = df["MedHouseVal"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Preprocessing Completed.")
```

```
Dataset Shape: (20640, 9)

First 5 Rows:
    MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  MedHouseVal
0    -122.23        4.526
1    -122.22        3.585
2    -122.24        3.521
3    -122.25        3.413
4    -122.25        3.422

Missing Values:
 MedInc         0
HouseAge       0
AveRooms       0
AveBedrms      0
Population     0
AveOccup       0
Latitude       0
Longitude      0
MedHouseVal    0
dtype: int64
```




```
Preprocessing Completed.
```

```
# Additional code cell

# Practical 14 - Code 2: Model Comparison

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Models
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(random_state=42)
}
```

```
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    print(f"\n{name}")
    print("RMSE:", rmse)
    print("R2 Score:", r2)
```

Enab
notifi
to ge
execu

```
Linear Regression
RMSE: 0.7455813830127763
R2 Score: 0.575787706032451

Decision Tree
RMSE: 0.7028289572288925
R2 Score: 0.6230424613065773

Random Forest
RMSE: 0.5051432839771741
R2 Score: 0.8052747336256919
```

```
# Practical 14 - Code 3: ML Pipeline

from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

# Create pipeline
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestRegressor(n_estimators=100, random_state=42))
])

# Train
pipeline.fit(X_train, y_train)

# Predict
y_pred = pipeline.predict(X_test)

# Evaluate
print("Pipeline R2 Score:", r2_score(y_test, y_pred))

# Example prediction (deployment simulation)
sample = X_test.iloc[:1]
prediction = pipeline.predict(sample)
print("Predicted House Price:", prediction)
```

```
Pipeline R2 Score: 0.8052747336256919
Predicted House Price: [0.5095]
```